MATH300: Mathematical Computing · Fall 2026

Project: Inside the Black Box, Algorithms Behind the Libraries

Groups must be formed, and project titles submitted on Canvas, by **October 26, 2026**. The project has two graded deliverables and an oral exam: a 3–5 page report for Project 2 (due **November 20, 2026**), a poster for the final project (due **December 11, 2026**), and an oral exam held between November 30 and December 17, 2026, 3:30 pm. Read the general rules below before you start; they apply throughout the project.

General remarks

- Projects are set for a group of **two or three** students.
- Form your group by **October 26, 2026**, and submit your project title as a group assignment on **Canvas** by the same date.
- Every group member must carry an **equal portion** of the coding and of the report and poster preparation.
- The scope can be adjusted to the number of participants, their interests and prior knowledge.
- Only **Python** may be used for programming.
- Implementation, documentation, validation and demonstration examples are all part of the project.
- Within these rules you are free to choose the implementation, code structure and exact content. You can discuss this with me at any time.

Version control and GitHub

Each group keeps **one GitHub repository for the whole project** (Project 2 and the final project), and every member commits from their own GitHub account.

- Use git from the very beginning. Commits should be small, consistent and clearly described. Avoid large commits.
- Each group member uploads their own code using **their own GitHub account**. Commits from a shared or borrowed account will not be credited to the other members.
- The repository name is up to your group.
- If the repository is private, add my GitHub account `say-yas` as a collaborator.
- The repository must contain a README describing the project, how to install and run the code, and the AI-use statement (see below).

Use of AI tools

AI-generated code is not accepted in any form. Code produced by a chat assistant or code generator (e.g. ChatGPT, Claude, Gemini) and copied, pasted or adapted into your project will be treated as work that is not your own.

- **Permitted:** inline coding assistance such as GitHub Copilot autocompletion while you write the code yourself, **provided its use is acknowledged**.
- **Acknowledgement:** include an AI-use statement in the README and in the report that names the tool(s) used and the parts of the code (files or functions) where they assisted. If no AI tools were used, state this explicitly.
- **Reports and posters must not be generated by AI** in any form: their text, figures and layout must be your own work. If AI tools were used for any other non-code purpose (e.g. checking grammar or searching the literature), describe this in the same statement.
- You are responsible for every line in your repository. In the oral exam you must be able to explain and modify any part of the code you submitted.

Deliverables and deadlines

Submit each deliverable following the details provided on the **Canvas** page. The written report and poster are handed in on **Crowdmark**, and the link to your GitHub repository is submitted separately on **Canvas**.

Deliverable	**Due**	**Submission**	**Notes**
Group and project title	Monday, October 26, 2026	Group assignment on Canvas	Group of two or three members; submit the project title.
Project 2 report	Friday, November 20, 2026	Report (PDF) on Crowdmark; repository link on Canvas	3–5 pages; the page limit **includes** references, figures and appendices.
Final project poster	Friday, December 11, 2026	Poster (PDF) on Crowdmark; repository link on Canvas	36 inches high by 48 inches wide (landscape). Target audience: undergraduate classmates with a basic background in math, physics or engineering.
Oral exam	Between November 30 and December 17, 2026, 3:30 pm	Individual meeting; details discussed in class	30 minutes per student. Your poster must already be submitted on Crowdmark before your exam. The last possible slot ends December 17 at 3:30 pm.

Oral exam

The oral exam is individual and lasts 30 minutes per student. It takes place between November 30 and December 17, 2026, 3:30 pm. The oral exams and how to arrange them will be discussed in class before the Thanksgiving break.

Before your oral exam, your poster must also be submitted on **Crowdmark**; that is, you must have finished both Project 2 and the final project. In the meeting you will:

- Present your results and algorithms.
- Possibly live-code and run parts of your algorithms to generate outputs on the spot.
- Answer questions about the theory, your implementation and your results. Each member should be able to explain the whole project, not only their own parts.

Bring a laptop with the repository cloned and a working environment, so that the code runs without additional setup.

Grading

Component	Points
Group and project title (on time)	10
Project 2 report	100
Final project poster	100
Oral exam (individual)	50
Bonus: extra steps beyond the required tasks	up to 10

The Project 2 report and the poster are each graded out of 100. What matters most is:

- **Coherence:** a clear story from the question, through the methods, to the results and conclusions.
- **Correctness:** of the mathematics, the code and the results.
- **Depth of analysis and discussion:** explain what your results mean, why they look the way they do, and what their limitations are, rather than only showing them.
- **Professional presentation:** figures with labeled axes and readable font sizes, captions and legends, a clean and readable layout, and proper references.

A complete, correct and well-discussed solution of the required tasks can earn full marks. Tasks marked **(Bonus)** and the suggestions under "Going further" are optional; exploring such extra steps can earn **up to 10 bonus points** in total.

Requirements for the report and the poster

Both the report and the poster must contain a clear description of the project, references, properly labeled figures and an author-contributions statement.

Content

- A clear and concise description of the problem, the methods and the results.
- The AI-use statement described above.
- The report should not contain any significant amount of code; refer to the repository instead.

Poster

- The poster must be **self-contained**: a reader should be able to follow it without the report, the code or a spoken explanation.
- You may be selective in what you show and how you narrate your story, so that an undergraduate audience with a basic background in math, physics or engineering can follow it.

- You may choose your own title for the poster; be creative.

References

- Include a list of references at the end of the report and the poster.
- Cite sources clearly where they are used in the text (papers, books, software, code repositories and data sets).
- Use one consistent citation style throughout.

Figures

- Every plot must have labeled x and y axes (and z axis for 3D plots or color bars), including units where applicable.
- Axis labels, tick labels and legends must have a font size similar to the surrounding text, so they remain readable without zooming.
- Each figure needs a legend when it shows more than one data series, and a caption that states what is shown.
- Use log scales where appropriate (e.g. for convergence plots) and say so in the caption.

Author contributions

Each group must include an author-contributions statement in the report and the README that states:

- Who did what: which parts of the code, experiments, analysis, writing and poster each member worked on.
- To what extent: the approximate share of each member in each part (e.g. 50 % / 50 %).
- Who took the lead in which parts.

The coding and the report and poster preparation must be shared equally between the members. The commit history should be consistent with the stated contributions.

Codebase requirements

- Well structured and documented, both in the code (docstrings, comments) and in the repository (README).
- Validated: show that your implementation produces correct results, e.g. on examples with known solutions.

Project description

Goal

In class we have used library functions to interpolate data, find roots and decompose matrices, usually without looking at what happens inside them. In this project you will open these black

boxes. You will choose **one theme** from the list below (for example interpolation) and **at least two methods** within that theme, explain how each one works, present its pseudocode, implement it yourself in Python, and compare your implementation with the version available in a package such as NumPy or SciPy [1].

Possible themes and methods

Choose **one theme** from the following list and **at least two methods** within it. You may also propose other methods or another theme related to the course (please check them with me first).

- **Interpolation:** cubic spline interpolation [4]; PCHIP, the monotone piecewise cubic Hermite interpolation of Fritsch and Carlson [5]; Akima interpolation [6]. SciPy: `CubicSpline`, `PchipInterpolator`, `Akima1DInterpolator` [2].
- **Root finding:** Ridders' method [7]; Brent's method [8]; the TOMS 748 algorithm of Alefeld, Potra and Shi [9]. SciPy: `ridder`, `brentq`, `toms748` [3].
- **Matrix decompositions:** QR decomposition by (modified) Gram–Schmidt or Householder reflections; LU decomposition with partial pivoting; Cholesky decomposition; SVD, for example by the one-sided Jacobi method [10, 11]. NumPy and SciPy: `numpy.linalg.qr`, `scipy.linalg.lu`, `numpy.linalg.cholesky`, `numpy.linalg.svd` [12].

Background

All three root-finding methods above start from an interval $[a, b]$ on which f changes sign, so that it contains a root, and shrink this interval step by step. The simplest such method, bisection, halves the interval in every step; the methods above use cleverer guesses (for example by fitting a curve through known points) to converge much faster, while still keeping the root safely inside the interval.

Interpolation constructs a function that passes exactly through given data points (x_i, y_i). The three interpolation methods above all use a cubic polynomial between each pair of neighboring points; they differ in how they choose the slopes at the data points. A cubic spline makes the whole curve twice continuously differentiable, which gives a very smooth curve but can overshoot. PCHIP chooses the slopes so that the curve does not overshoot monotone data, and Akima's method chooses them from nearby points only, which reduces wiggles near outliers.

A matrix decomposition writes a matrix as a product of simpler matrices, for example $A = QR$ with Q orthogonal and R upper triangular, $A = LU$ with L lower and U upper triangular, or $A = USV^{\mathsf{T}}$ (the SVD). These decompositions are the workhorses behind solving linear systems, least-squares fitting and data compression. Different algorithms for the same decomposition can behave very differently in floating-point arithmetic: for example, classical Gram–Schmidt can lose the orthogonality of Q, while Householder reflections keep it.

Tasks for Project 2 (report, due November 20, 2026)

For **each** of your (at least two) chosen methods:

1. Explain in your own words how the method works, and cite your sources.
2. Present its pseudocode. (Pseudocode is welcome in the report; Python code is not.)

3. Implement it yourself in Python. Do not call the library version inside your own implementation.
4. Check your implementation on a few examples with known answers, and compare its results with the library version.
5. **(Bonus)** Construct an example where one of your methods fails or struggles, and explain why.
6. **(Bonus)** Implement a third method from the same theme, and include it in the comparison in the final project.

Tasks for the final project (poster, due December 11, 2026)

1. Compare your methods with each other and with the library versions on several test problems: accuracy, and number of iterations or function evaluations where this applies. Present the results in plots or tables.
2. Discuss what you observe: when does each method work well, and when does it struggle? Which method would you recommend for which kind of problem? Why might the library version be faster or more robust than yours?
3. **(Bonus)** Compare the computing time of your implementations with the library versions.

Going further

These optional steps count toward the bonus points. For example, you can implement a method from another theme.

References

[1] SciPy documentation. https://docs.scipy.org/doc/scipy/

[2] SciPy. *Interpolation (scipy.interpolate)*. https://docs.scipy.org/doc/scipy/reference/interpolate.html

[3] SciPy. *Optimization and root finding (scipy.optimize)*. https://docs.scipy.org/doc/scipy/reference/optimize.html

[4] C. de Boor (2001). *A Practical Guide to Splines*, revised edition. Springer. https://link.springer.com/book/9780387953663

[5] F. N. Fritsch and R. E. Carlson (1980). Monotone piecewise cubic interpolation. *SIAM Journal on Numerical Analysis*, 17, 238–246. https://doi.org/10.1137/0717021

[6] H. Akima (1970). A new method of interpolation and smooth curve fitting based on local procedures. *Journal of the ACM*, 17, 589–602. https://doi.org/10.1145/321607.321609

[7] C. J. F. Ridders (1979). A new algorithm for computing a single root of a real continuous function. *IEEE Transactions on Circuits and Systems*, 26, 979–980. https://doi.org/10.1109/TCS.1979.1084580

[8] R. P. Brent (1973). *Algorithms for Minimization without Derivatives*. Prentice-Hall. https://maths-people.anu.edu.au/~brent/pub/pub011.html

[9] G. E. Alefeld, F. A. Potra and Y. Shi (1995). Algorithm 748: Enclosing zeros of continuous functions. *ACM Transactions on Mathematical Software*, 21, 327–344. https://na.math.kit.edu/alefeld/download/1995_Algorithm_748_Enclosing_Zeros_of_Continuous_Functions.pdf

[10] L. N. Trefethen and D. Bau (1997). *Numerical Linear Algebra*. SIAM. https://doi.org/10.1137/1.9780898719574

[11] G. H. Golub and C. F. Van Loan (2013). *Matrix Computations*, 4th edition. Johns Hopkins University Press. https://nhigham.com/2013/05/31/fourth-edition-of-matrix-computations/

[12] NumPy. *Linear algebra (numpy.linalg)*. https://numpy.org/doc/stable/reference/routines.linalg.html